

Task 1 — The Field Agent

Building the Rust endpoint agent: enroll, collect, obey orders, never leave the post.

Owner	Dev A
Project	IRONCLAW — Behavioral Threat Hunting Platform
Component	Endpoint Agent (Rust)
Integrates with	Dev B's backend + console, over the shared policy contract (§6)

1. The Story Behind Your Task

Picture a nationwide private security firm. The firm protects thousands of buildings scattered across the country. In every building stands one trained **security guard**. The guard watches the doors, logs who comes and goes, notices anything suspicious, and radios it back to headquarters. The guard never decides company-wide strategy — they faithfully follow the **post orders** handed down from HQ, and they keep watching even when the radio goes dead.

You are building that guard. In IRONCLAW the “building” is a Windows endpoint (a laptop, a server, a workstation), and the guard is a small, fast program written in Rust that lives on it — the **endpoint agent**. Your teammate (Dev B) is building HQ: the dispatch brain and the dispatcher's console where humans write the post orders. Your two pieces meet over a small, well-defined radio protocol that this document spells out.

The metaphor in one line

Dev B writes the post orders and decides who gets them. You build the guard who receives those orders, follows them to the letter, and never abandons the post.

2. Your Mission

Build the IRONCLAW endpoint agent: a Rust program that **enrolls** itself with the backend, **collects** security telemetry from the Windows endpoint, ships it back securely, and — most importantly — **obeys the policy** (the post orders) that the backend assigns to it. When HQ changes the orders, your agent must notice, apply the change to what it is actually doing, and remember the new orders even after a reboot.

Think of the work as four promises the guard makes:

- **I will report for duty.** On first run I introduce myself to HQ and receive my identity and my first set of orders.
- **I will watch what I am told to watch.** I collect exactly the telemetry my current orders specify — no more, no less.
- **I will check in regularly.** Every minute I radio HQ, report my status, and ask whether my orders have changed.
- **I will obey new orders the moment they arrive, and remember them.** A new policy reconfigures me live, and survives a restart.

3. Concepts You Need First

Telemetry

The raw observations the guard records: a process started, a network connection opened, a driver loaded, a user logged in. On Windows these come from sources like Sysmon, the Windows Security event log, PowerShell logging, and ETW providers. Each observation becomes a structured **event** that you ship to the backend.

Policy (the “post orders”)

A policy is the single document that tells the agent how to behave: which event types to collect, how fast it may ship them, which invariants (non-negotiable safety rules) are active, and a handful of lightweight on-site detection filters. The backend owns the policy; the agent only ever *receives* and *applies* it. Every policy carries a **version number** and a **hash** so both sides can tell instantly whether the agent is up to date.

Why versioning matters

Imagine every guard's post orders had a revision number stamped on the cover. When the guard radios in saying 'I'm on revision 41' and HQ is now on 42, HQ knows to read out the new orders. If both say 41, no breath is wasted. That stamped number is the policy version — cheap to compare, and the heart of staying in sync.

Heartbeat

The periodic radio check-in (about once a minute). The agent sends its health and its current policy version; the backend may reply with new orders. The heartbeat is how the agent stays alive in HQ's eyes and how it learns that its orders have changed without constant polling.

Invariants

A small set of safety rules that can never be switched off from the field — for example, always watch for someone trying to read the password vault (LSASS) or load an unsigned driver. Even if a policy tries to relax everything else, the invariants stand. They are the guard's “you may never leave these exits unwatched” rules.

4. Work Breakdown

Tackle these in order. Each milestone leaves the agent in a working, demonstrable state — you should be able to show progress after every one.

M1 — Enrollment & Identity

When the agent runs for the first time it must “report for duty.” It generates its own cryptographic key pair, then calls the registration endpoint with its hostname, OS details, and public key. HQ responds with a permanent **agent_id**, the backend’s public key, and the agent’s very first policy. The agent stores this identity locally so it never re-enrolls by accident.

- Generate and persist the agent key pair on first run.
- Call `POST /api/v1/agents/register` with hostname, OS info, agent version, and public key.
- Persist the returned `agent_id`, backend public key, and initial policy to local storage.
- If an identity already exists locally, skip enrollment and load it instead.

M2 — Telemetry Collection Engine

Build the collectors that turn Windows activity into structured events. Each collector should be independently switchable, because the policy will later turn them on and off. At minimum: process/network/driver events (Sysmon), authentication events (Security log), and PowerShell activity.

- One collector module per source, each reading a list of enabled event IDs from config.
- Normalize every observation into a common event structure with a timestamp, type, and payload.
- Make the set of active collectors a function of the current policy — not hard-coded.

M3 — Secure Transport & Local Buffering

The guard must not lose its notes if the radio drops. Events are first written to a local buffer (an in-memory ring plus a disk spool), then shipped to the backend in encrypted, compressed batches. If the network is down, events queue on disk and ship when the link returns.

- In-memory ring buffer for hot events, with overflow to an on-disk spool (size-capped).
- Batch events and encrypt them end-to-end before shipping to `POST /agents/{id}/events`.
- Retry with backoff on failure; never drop buffered events while disk space remains.

M4 — The Heartbeat Loop

Every ~60 seconds, radio in. Send the agent’s health (uptime, buffer depth, events shipped) and its **current policy version**. Read the response: if it contains a `policy_update`, hand it to the policy engine (M5). This loop is the agent’s lifeline and its update channel in one.

- Send `POST /agents/{id}/heartbeat` on a fixed interval, including current policy version.
- Parse the response for an optional newer policy.
- Keep beating even while events are being collected and shipped — never block one on the other.

M5 — Policy Enforcement (apply live)

This is the centerpiece. When a new policy arrives, the agent must actually **change its behavior** — not just log that an update was available. If the new policy enables DLL events, the DLL collector starts. If it lowers the rate limit, shipping slows. If it disables a collector, that collector stops. The transition must be live, with no restart required.

- Validate the incoming policy (check the hash; reject malformed orders).
- Diff it against the running configuration and apply each change to the live collectors, rate limiters, and filters.
- Always keep invariants active regardless of what the rest of the policy says.
- Record the new policy version as the agent's current version so the next heartbeat reports it.

The trap to avoid

A half-built guard nods at the new orders and then keeps doing exactly what it did yesterday. 'Update available' in a log file is not enforcement. The test of this milestone is behavioral: flip a setting at HQ and watch the endpoint's actual collection change within one heartbeat.

M6 — Policy Persistence & Versioning

Orders must survive a reboot. The current policy is written to disk the moment it is applied. On startup the agent loads its stored policy and configures itself from it *before* the first heartbeat — so a rebooted endpoint comes back already doing the right thing, even if HQ is briefly unreachable.

- Persist the active policy (and its version/hash) to disk atomically on every apply.
- On startup, load and apply the stored policy before the first network call.
- Report the loaded version in the first heartbeat so HQ can push an update if one was missed while offline.

M7 — Resilience

The guard keeps the post even when the radio is dead. Define behavior for: backend unreachable (buffer and retry), malformed policy (keep the last good one, report the rejection), and clean shutdown (flush the buffer, persist state).

5. Use-Case Scenarios

These narratives describe the behavior we will demo and test. Read them as the “definition of working.”

Scenario A — First day on the job

A fresh endpoint boots with the agent installed. The agent finds no local identity, so it generates its keys and registers with HQ. HQ replies with agent_id a1b2... and policy **v1**. The agent saves both, spins up the collectors v1 specifies, and begins shipping events. Within a minute it sends its first heartbeat reporting ‘I am on v1, healthy.’

Scenario B — HQ tightens the orders

A SOC analyst at HQ decides finance servers should now also collect DLL-load events. Dev B's console compiles a new policy, **v2**, and assigns it. On this endpoint's next heartbeat (‘I am on v1’), HQ replies with v2. The agent validates it, sees that `dll_events_enabled` flipped to true, starts the DLL collector *live*, writes v2 to disk, and the following heartbeat now reports ‘I am on v2.’ No reboot, no reinstall.

Scenario C — The radio goes dead

A network outage cuts the endpoint off from HQ for two hours. The guard keeps watching: events pile into the disk spool, heartbeats fail quietly and retry. When the link returns, the buffered events ship in order and the heartbeat resumes. Nothing observed during the outage is lost.

Scenario D — A reboot mid-shift

The endpoint is on v2 when it reboots for patching. On startup the agent reads its stored policy, configures the DLL collector and rate limits from v2 *before* touching the network, and only then heartbeats. The endpoint never reverts to default behavior, even for a moment.

6. The Shared Contract (your radio protocol)

This is the seam between you and Dev B. Agree on it together on day one and treat it as fixed; both sides build against it independently. You are the **consumer** of the policy — you never invent its shape, you only read and obey it.

Endpoints you call

```
POST /api/v1/agents/register
  -> agent introduces itself, receives agent_id + initial policy

POST /api/v1/agents/{agent_id}/heartbeat (every ~60s)
  -> agent reports health + current policy version
  <- backend replies with policy_update IF a newer version exists

GET /api/v1/agents/{agent_id}/policy?current_version=41
  -> "give me my orders if newer than v41"
  <- { "status": "update_available", "policy": { ...schema above... } }
      or { "status": "current" }
```

The policy document you receive and must honor

```

{
  "version": 42,                                // integer, increases on every change
  "hash": "sha256:9f86d08...",                 // fingerprint of the compiled policy
  "invariants": {                               // hard safety rules, cannot be disabled on-site
    "lsass_access": { "enabled": true },
    "unsigned_drivers": { "enabled": true },
    "credential_dumping": { "enabled": true }
  },
  "collection": {                               // what telemetry to gather
    "sysmon_events": [1, 3, 7, 8, 10, 11, 12, 13, 22, 23],
    "security_events": [4624, 4625, 4672, 4688, 4720],
    "powershell_logging": true,
    "dll_events_enabled": false,
    "file_events": { "enabled": true, "path_filters": ["C:\\Windows\\Temp\\*"] }
  },
  "rate_limits": {                             // protect the network and the backend
    "events_per_second": 1000,
    "network_events_per_second": 200
  },
  "detection_filters": [                       // lightweight on-site rules
    { "name": "suspicious_powershell", "match": "cmdline",
      "pattern": "-enc", "action": "flag" }
  ]
}

```

Contract discipline

If you find yourself wishing the policy had a field it doesn't, do not quietly add it on the agent side. Raise it with Dev B and change the contract together. The whole system's reliability rests on both halves reading the same sheet of music.

7. Definition of Done

Your task is complete when every line below is demonstrably true:

#	Acceptance criterion
1	A fresh agent enrolls automatically and persists its identity; it never re-enrolls on restart.
2	The agent collects the exact event types named in its current policy, and nothing it was told to disable.
3	Events buffer to disk during an outage and ship in order when connectivity returns; none are lost.
4	A heartbeat fires on a fixed interval and reports the agent's current policy version.
5	When HQ assigns a new policy, the agent applies the change to live behavior within one heartbeat — verified by observing the endpoint, not by reading a log.
6	Invariants remain active no matter what the rest of the policy says.
7	The active policy is persisted and re-applied on startup before the first network call; a reboot does not revert behavior.

#	Acceptance criterion
8	A malformed or failed-hash policy is rejected; the last good policy stays in force and the rejection is reported.

8. Out of Scope (Dev B owns these)

- Authoring policies or deciding their contents — you only receive them.
- Grouping devices or deciding which endpoint gets which policy.
- The console/UI humans use to write orders.
- Storing fleet-wide state or computing analytics across endpoints.

9. Glossary

Term	Plain meaning
Agent	The Rust program on each endpoint — the security guard.
Endpoint	A protected Windows machine — the building.
Policy	The post orders: what to collect and how to behave.
Heartbeat	The ~60s radio check-in that also delivers new orders.
Invariant	A safety rule that can never be turned off in the field.
Enrollment	Reporting for duty: first-run registration with HQ.
Telemetry / event	A single recorded observation shipped to the backend.